O&G · 6 — Discovering the decline law itself from production data

Every earlier notebook in this series *assumed* a model form (Arps, a Fetkovich aquifer, the diffusivity equation) and either fit its parameters or filled a gap with a neural network. This closing notebook asks a more basic question: given nothing but a rate-vs-time history, can we discover the governing decline **equation** itself, with no assumed functional form at all?

We use **SINDy** (`06_sindy_model_discovery.jl`) on a well whose true (but, to the algorithm, unknown) behavior is harmonic decline ($b = 1$ in the Arps family), which happens to correspond to the clean autonomous ODE $\dot{q} = -\frac{D_i}{q_i} q^2$ — a single quadratic term. If SINDy is given a library rich enough to contain that term, sparse regression should isolate it automatically.

```
using DataDrivenDiffEq, DataDrivenSparse, ModelingToolkit, OrdinaryDiffEq, Random, Plots
```

1. A harmonically declining well

$$\dot{q} = -\frac{D_i}{q_i} q^2$$

```
harmonic_decline! (generic function with 1 method)
```

```
function harmonic_decline!(du, u, p, t)
    Di, qi = p
    q = u[1]
    du[1] = -Di * q^2 / qi
    return nothing
end
```

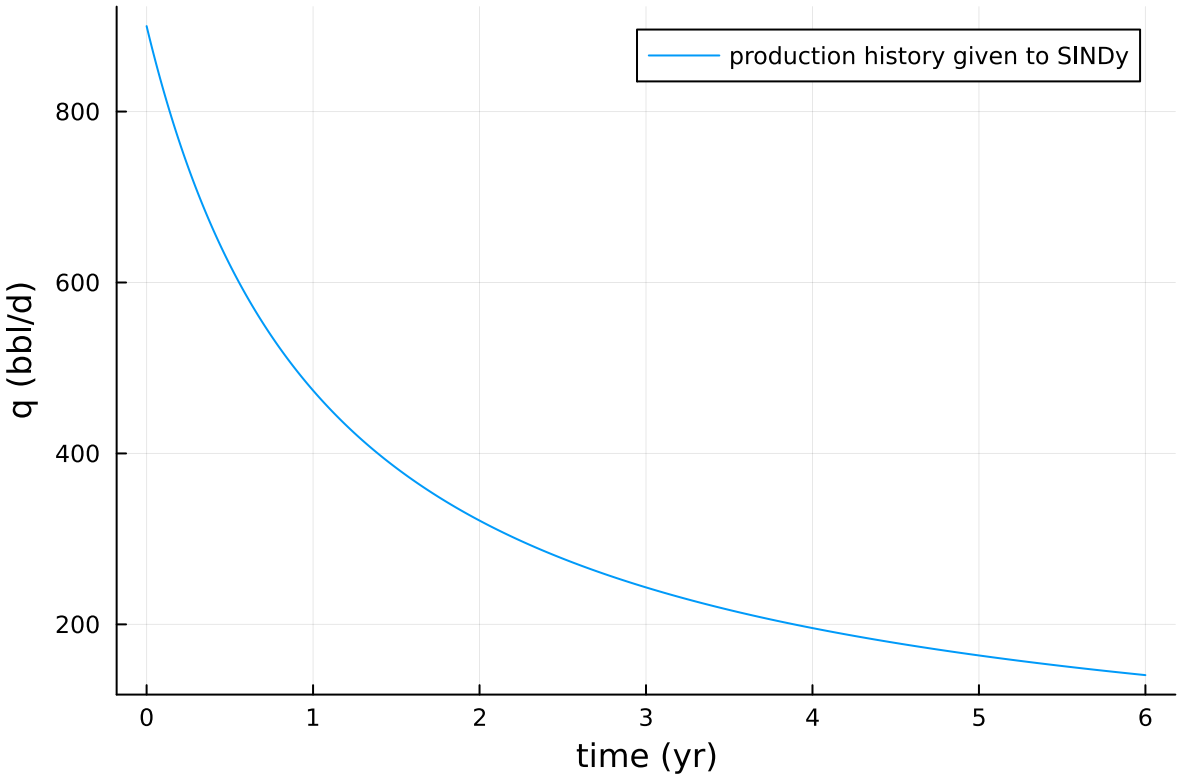
```
[0.0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, (
```

```
begin
    rng_og6 = Random.default_rng()
    Random.seed!(rng_og6, 9)

    qi_og6, Di_og6 = 900.0, 0.9   # bbl/d, 1/yr
    tspan_og6 = (0.0, 6.0)
    saveat_og6 = 0.01

    well_prob = ODEProblem(harmonic_decline!, [qi_og6], tspan_og6, (Di_og6, qi_og6))
    well_sol = solve(well_prob, Tsit5(); saveat=saveat_og6)

    Xq = Array(well_sol)
    tq = well_sol.t
end
```




```
plot(tq, Xq[1, :]; xlabel="time (yr)", ylabel="q (bbl/d)", label="production history given to SINDy")
```

2. Build the `DataDrivenProblem`

As in the general SINDy notebook, we supply the derivative directly here (computable because we control the synthetic example); in practice `DataDrivenDiffEq.jl` can also estimate it from the trajectory by finite differencing or collocation.

```
Continuous DataDrivenProblem{Float64} ##DDProblem#296 in 1 dimensions and 601 samples
```

```
begin
    DXq = similar(Xq)
    for (i, ui) in enumerate(eachcol(Xq))
        harmonic_decline!(view(DXq, :, i), ui, (Di_og6, qi_og6), tq[i])
    end
    ddprob_q = ContinuousDataDrivenProblem(Xq, tq; DX=DXq)
end
```

3. A polynomial candidate library

We deliberately do *not* tell SINDy the true exponent — the library contains every monomial in q up to degree 3, and sparse regression must pick out the one that actually matters.

```
Model ##Basis#301 with 4 equations
States : q
Independent variable: t
Equations
φ₁ = 1
φ₂ = q
φ₃ = q^2
φ₄ = q^3
```

```
begin
    @variables q
    decline_basis = Basis(polynomial_basis([q], 3), [q])
end
```

```
⚠ Attempting to print a symbolic expression in a Pluto notebook. Please run `import L
  atexify` to enable pretty-printing of symbolic expressions. This warning will only
  display once.
```

```
Model ##Basis#308 with 1 equations
States : q
Parameters : p₁
Independent variable: t
Equations
Differential(t, 1)(q) = p₁*(q^2)
```

```
begin
    sparse_opt_q = STLSQ(exp10.(-6:0.25:1))
    sindy_res_q = solve(ddprob_q, decline_basis, sparse_opt_q)
    sindy_decline_law = get_basis(sindy_res_q)
end
```

```
println(sindy_decline_law)
```

```
Model ##Basis#308 with 1 equations
States : q
Parameters : p₁
Independent variable: t
Equations
Differential(t, 1)(q) = p₁*(q^2)
```

Error message

MethodError: no method matching get_parameter_values(::DataDrivenDiffEq.DataDrivenSolution{Float64})

The function `get_parameter_values` exists, but no method is defined for this combination of argument types.

Closest candidates are:

```
  get_parameter_values(::DataDrivenDiffEq.Basis)
   @ DataDrivenDiffEq ~/.julia/packages/DataDrivenDiffEq/YVjFe/src/basis/type.jl:642
```

Show stack trace...

```
1 get_parameter_values(sindy_res_q)
```

4. Validate the discovered law by forecasting forward

Error message

```
MethodError: no method matching
get_parameter_values(::DataDrivenDiffEq.DataDrivenSolution{Float64})
The function `get_parameter_values` exists, but no method is defined for this
combination of argument types.

Closest candidates are:
  get_parameter_values(::DataDrivenDiffEq.Basis)
   @ DataDrivenDiffEq ~/.julia/packages/DataDrivenDiffEq/YVjFe/src/basis/type.jl:642
```

Show stack trace...

```
1 begin
2     discovered_well_prob = ODEProblem(sindy_decline_law, [qi_og6], tspan_og6,
      get_parameter_values(sindy_res_q))
3     discovered_well_sol = solve(discovered_well_prob, Tsit5(); saveat=saveat_og6)
4
5     plot(tq, Xq[1, :]; label="true harmonic decline", lw=2, xlabel="time (yr)",
      ylabel="q (bbl/d)")
6     plot!(tq, Array(discovered_well_sol)[1, :]; label="SINDy-discovered model", lw=2,
      ls=:dash)
7 end
```

Takeaways

- Given a rich-enough candidate library, SINDy recovered that this well's decline is governed by a single q^2 term — i.e. it independently rediscovered harmonic ($b = 1$) Arps decline directly from the rate history, without that model form ever being stated.
- A non-integer decline exponent ($0 < b < 1$, the common hyperbolic case) is not a single monomial, so a plain polynomial library would only approximate it; a production application would extend the library with fractional powers of q or apply SINDy locally over the operating range, but the mechanics — build a library, run sparse regression, validate by simulating forward — stay identical.
- This closes the loop on the notebook series: `01_decline_curve_analysis.jl` assumed the Arps form and fit its parameters; this notebook shows that, at least for the clean cases, the form itself

doesn't have to be assumed either.